

阿里

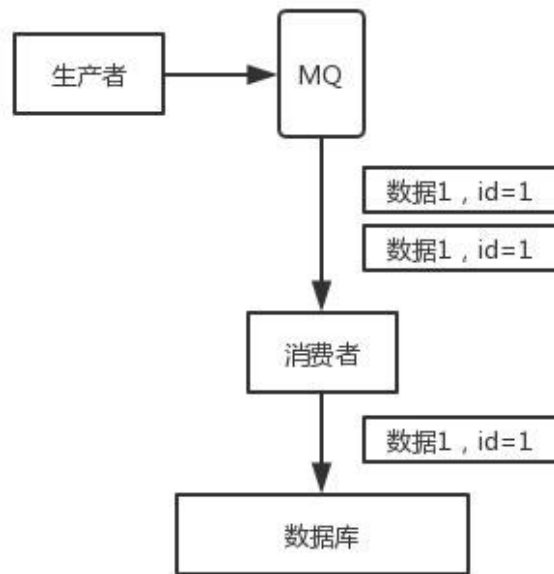
消息队列的消费幂等性如何保证？

🔥 薪资：40K

★ 岗位：高级开发

消费幂等性问题

举个例子：假设你有个系统，消费一条消息就往数据库里插入一条数据，要是你一条消息重复两次，你不就插入了两条，这数据不就错了？但是你要是消费到第二次的时候，自己判断一下是否已经消费过了，若是就直接扔了，这样不就保留了一条数据，从而保证了数据的正确性。



简单的消息去重解决方案

例如：假设我们业务的消息消费逻辑是：插入某张订单表的数据，然后更新库存：

```
insert into t_order values .....
update t_inv set count = count-1 where good_id = 'good123';
```

要实现消息的幂等，我们可能会采取这样的方案：

```
select * from t_order where order_no = 'order123'
if(order != null) {
    return ;//消息重复，直接返回
}
```

这对于很多情况下，的确能起到不错的效果，但是在并发场景下，还是会有问题。

并发重复消息

假设这个消费的所有代码加起来需要1秒，有重复的消息在这1秒内（假设100毫秒）内到达（例如生产者快速重发，Broker重启等），那么很可能，上面去重代码里面会发现，数据依然是空的（因为上一条消息还没消费完，还没成功更新订单状态），

那么就会穿透掉检查的挡板，最后导致重复的消息消费逻辑进入到非幂等安全的业务代码中，从而引发重复消费的问题（如主键冲突抛出异常、库存被重复扣减而没释放等）

```
select * from t_order where order_no = 'order123'  
if(order != null) {  
    return ;//消息重复，直接返回  
}
```

并发去重的解决方案之一

要解决上面并发场景下的消息幂等问题，一个可取的方案是开启事务把select 改成 select for update语句，把记录进行锁定。

```
select * from t_order where order_no = 'THIS_ORDER_NO' for update  //开启事务
if(order.status != null) {
    return ;//消息重复，直接返回
}
```

但这样消费的逻辑会因为引入了事务包裹而导致整个消息消费可能变长，并发度下降。

当然还有其他更高级的解决方案，例如更新订单状态采取乐观锁，更新失败则消息重新消费之类的。但这需要针对具体业务场景做更复杂和细致的代码开发、库表设计，不在本次探讨的范围。

但无论是select for update，还是乐观锁这种解决方案，实际上都是基于业务表本身做去重，这无疑增加了业务开发的复杂度，一个业务系统里面很大部分请求处理都是依赖MQ的，如果每个消费逻辑本身都需要基于业务本身而做去重/幂等的开发的话，这是繁琐的工作量。本次主要分享一些通用的消息幂等处理的方法，从而抽象出一定的工具类用以适用各个业务场景。

如何保证消息只消费成功一次？

在消息中间件里，有一个投递语义的概念，而这个语义里有一个叫“Exactly Once”，即消息肯定会被成功消费，并且只会被消费一次。以下是阿里云里对Exactly Once的解释：

Exactly-Once 是指发送到消息系统的消息只能被消费端处理且仅处理一次，即使生产端重试消息发送导致某消息重复投递，该消息在消费端也只被消费一次。

在我们业务消息幂等处理的领域内，可以认为业务消息的代码肯定会被执行，并且只被执行一次，那么我们可以认为是Exactly Once。

但这在分布式的场景下想找一个通用的方案几乎是不可能的。不过如果是针对基于数据库事务的消费逻辑，实际上是可行的。

基于关系数据库事务插入消息表

假设我们业务的消息消费逻辑是：更新MySQL数据库的某张订单表的状态：

```
update t_order set status = 'SUCCESS' where order_no= 'order123';
```

要实现Exactly Once即这个消息只被消费一次（并且肯定要保证能消费一次），我们可以这样做：在这个数据库中增加一个消息消费记录表，把消息插入到这个表，并且把原来的订单更新和这个插入的动作放到同一个事务中一起提交，就能保证消息只会被消费一遍了。

1. 开启事务
2. 插入消息表（处理好主键冲突的问题）
3. 更新订单表（原消费逻辑）
4. 提交事务

说明：这时候如果消息消费成功并且事务提交了，那么消息表就插入成功了，这时候就算RocketMQ还没有收到消费位点的更新再次投递，也会插入消息失败而视为已经消费过，后续就直接更新消费位点了。这保证我们消费代码只会执行一次。如果事务提交之前服务挂了（例如重启），对于本地事务并没有执行所以订单没有更新，消息表也没插入成功；而对于RocketMQ服务端来说，消费位点也没更新，所以消息还会继续投递下来，投递下来发现这个消息插入消息表也是成功的，所以可以继续消费。这保证了消息不丢失。

消息表方案局限性

基于这种方式，的确这是有能力拓展到不同的应用场景，因为他的实现方案与具体业务本身无关——而是依赖一个消息表。

但是这里有它的局限性：

1. 消息的消费逻辑必须是依赖于关系型数据库事务。如果消费的消费过程中还涉及其他数据的修改，例如 Redis 这种不支持事务特性的数据源，则这些数据是不可回滚的。
2. 数据库的数据必须是在一个库，跨库无法解决。

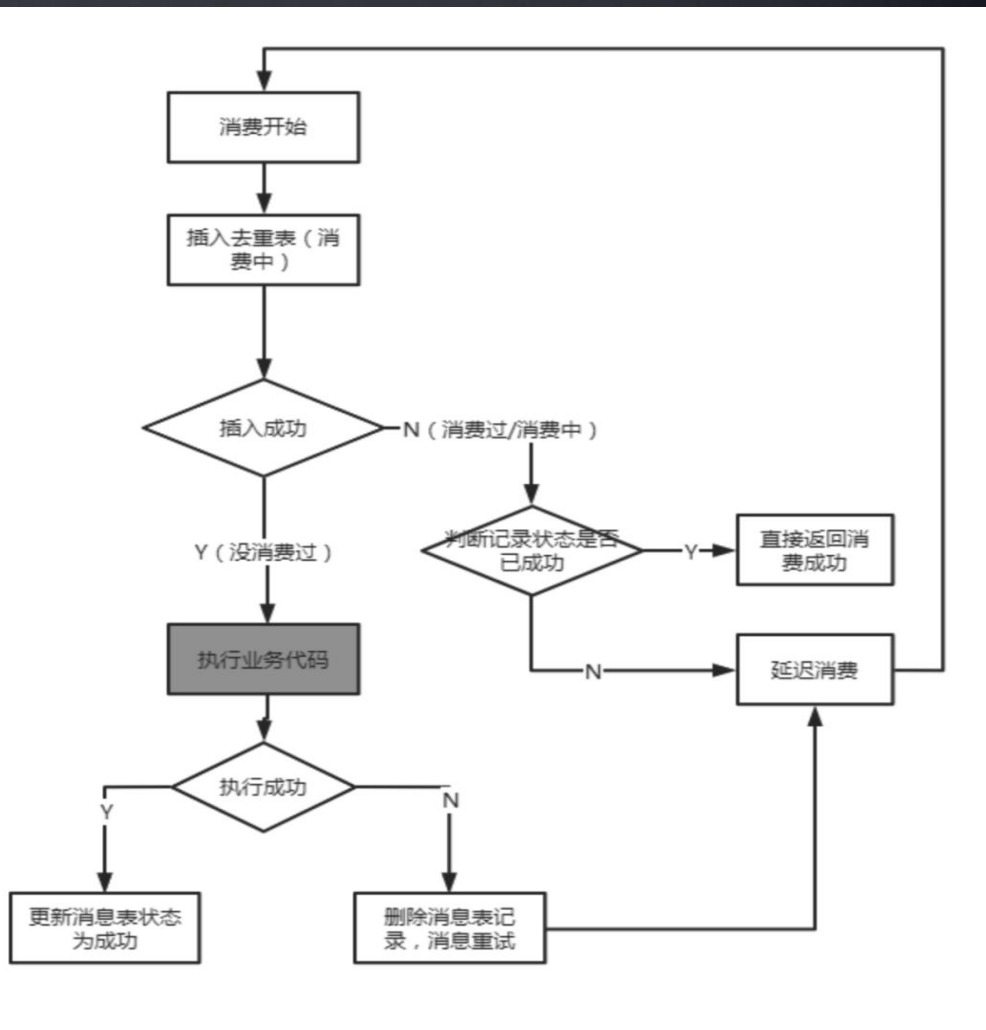
注：业务上，消息表的设计不应该以消息ID作为标识，而应该以业务的业务主键作为标识更为合理，以应对生产者的重发。阿里云上的消息去重只是 RocketMQ 的 `messageId`，在生产者因为某些原因手动重发（例如上游针对一个交易重复请求了）的场景下起不到去重/幂等的效果（因消息id不同）。

基于消息幂等表的非事务方案

上面消息表+本地事务的方案之所以有其局限性和并发的短板，究其根本是因为它依赖于关系型数据库的事务，且必须要把事务包裹于整个消息消费的环节。如果我们能不依赖事务而实现消息的去重，那么方案就能推广到更复杂的场景例如：RPC、跨库等。

右图是去事务化后的消息幂等方案的流程，可以看到，**此方案是无事务的，而是针对消息表本身做了状态的区分：消费中、消费完成。**只有消费完成的消息才会被幂等处理掉。

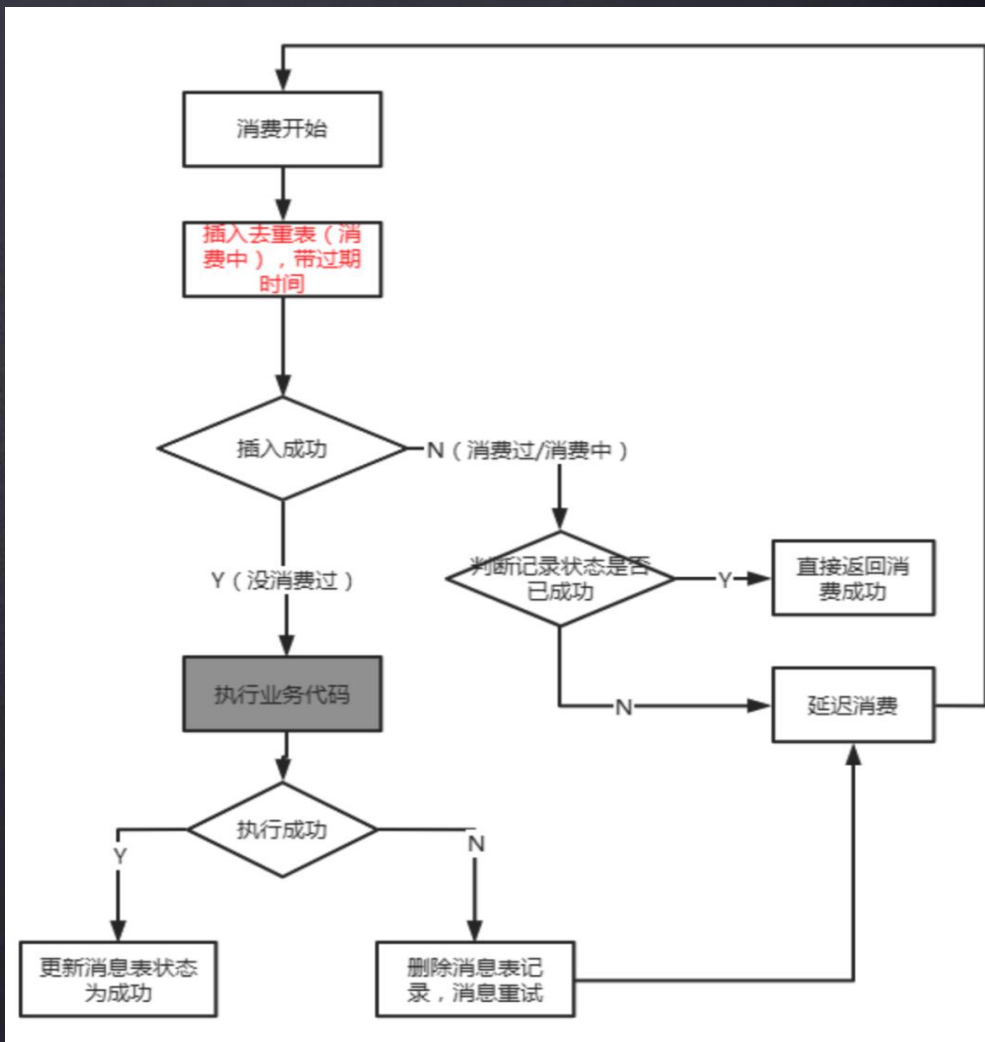
而对于已有消费中的消息，后面重复的消息会触发延迟消费（在RocketMQ的场景下即发送到RETRY TOPIC），之所以触发延迟消费是为了控制并发场景下，第二条消息在第一条消息没完成的过程中，去控制消息不丢（如果直接幂等，那么会丢失消息（同一个消息id的话），因为上一条消息如果没有消费完成的时候，第二条消息你已经告诉broker成功了，那么第一条消息这时候失败broker也不会重新投递了）



非事务方案存在消息丢失的风险

在并发场景下我们依赖于消息状态是做并发控制使得第2条消息重复的消息会不断延迟消费（重试）。但如果这时候第1条消息也由于一些异常原因（例如机器重启了、外部异常导致消费失败）没有成功消费成功呢？也就是说这时候延迟消费实际上每次下来看到的都是消费中的状态，最后消费就会被视为消费失败而被投递到死信Topic中（RocketMQ默认可以重复消费16次）。

解决方案：插入的消息表必须要带一个最长消费过期时间，例如10分钟，意思是如果一个消息处于消费中超过10分钟，就需要从消息表中删除（需要程序自行实现）。所以最后这个消息的流程如右图



更灵活的消息表存储媒介

我们这个方案实际上没有事务的，只需要一个存储的中心媒介，那么自然我们可以选择更灵活的存储媒介，例如Redis。使用Redis有两个好处：

1. 性能上损耗更低
2. 上面我们讲到的超时时间可以直接利用Redis本身的ttl实现

当然Redis存储的数据可靠性、一致性等方面是不如MySQL的，需要根据实际情况取舍。

以下是一个利用Redis去重示例，用以业务中如果使用此工具加入消息去重幂等的是多么简单：

来自：<https://github.com/Jaskey/RocketMQDedupListener>

```
//利用Redis做幂等表
DefaultMQPushConsumer consumer = new DefaultMQPushConsumer("TEST-APP1");
consumer.subscribe("TEST-TOPIC", "*");

String appName = consumer.getConsumerGroup(); // 大部分情况下可直接使用consumer group名
StringRedisTemplate stringRedisTemplate = null; // 这里省略获取StringRedisTemplate的过程
DedupConfig dedupConfig = DedupConfig.enableDedupConsumeConfig(appName,
stringRedisTemplate);
DedupConcurrentListener messageListener = new SampleListener(dedupConfig);

consumer.registerMessageListener(messageListener);
consumer.start();
```

总结

1. 简单的消息去重解决方案：先判断再更新，但是会有并发重复消费的问题
2. 并发去重的解决方案：使用select for update悲观锁(导致并发度下降)、乐观锁(增加业务复杂度)
3. 如何保证消息只消费成功一次(Exactly Once)? 比如基于关系数据库事务插入消息表，但是局限性是依赖关系型数据库事务、不能跨库、Redis这种不支持事务的数据源，无法回滚
4. 基于消息幂等表的非事务方案：优点是去事务化，缺点是存在消息丢失的风险
5. 消息丢失解决方案：消息表增加一个最长消费过期时间，例如10分钟，意思是一个消息处于消费中超过10分钟，就需要从消息表中删除（需要程序自行实现）
6. 考虑到消息表存储性能问题，可以考虑使用redis作为存储媒介，但是数据可靠性、一致性等方面是不如MySQL，需要根据实际情况取舍